PilotFish Database (SQL) Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **database worker inside your route**. The transaction carries XML in SQLXML format (or a simple SQL query you configure). This processor opens a JDBC connection, runs the SQL/XML against your database, and puts the XML result back on the transaction — optionally triggering another listener with the output.

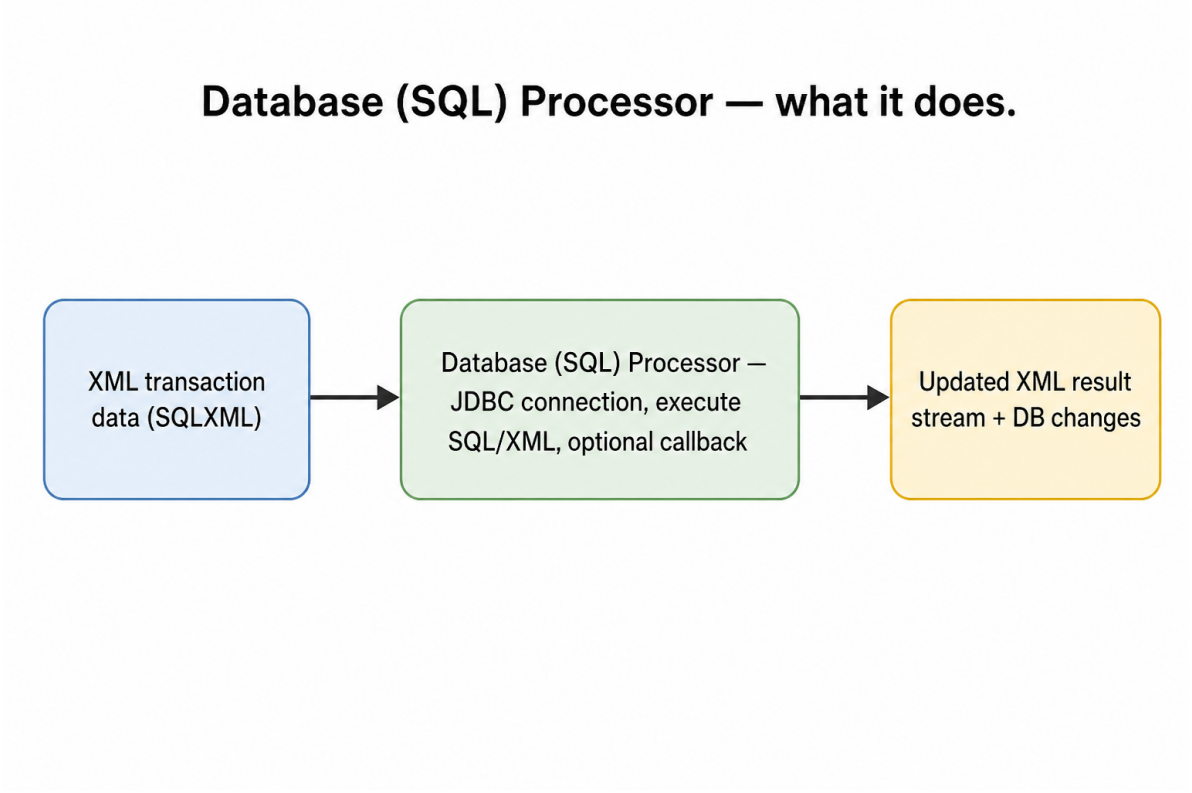

Figure 1: At a glance (plain English)

- **Executes** SQLXML documents from the transaction stream or a configured input file.
- **Connects** via JDBC URL or a server data source.
- **Supports** ad hoc **Write Query** mode that builds SQLXML from SQL + parameters.

- **Optional** callback listener for programmatic handling of query results.

Purpose

This document is a **deep dive** into the **Database (SQL)** processor as shipped in `eip.war.hs.26R1.11`. The processor class is a thin wrapper; all behavior lives in `DatabaseSqlExecution` and `SQLXMLHandler`.

Provenance	Value
WAR	`eip.war.hs.26R1.11`
Module JAR	`xcs-modules-db-1.0.1.jar`
Decompiler	CFR 0.152
Implementation class	`com.pilotfish.eip.modules.db.DatabaseSqlPr`
Execution engine	`com.pilotfish.eip.modules.db.DatabaseSqlEx`
Decompiled path	`war-pipeline/decompiled/eip-26R1.11/.../Da`
Processor type string	`Database (SQL)`
Description	`Translates incoming xml data into SQL statements which are used to populate/update a specified database.`
Categories	`SQL`, `DATABASE`

Related components:

Class	Role
`DatabaseSqlExecution`	Configuration descriptor, handler lifecycle, execute path
`SQLXMLHandler`	JDBC + SQLXML execution
`DatabaseSqlListener`	Listener counterpart (same execution class pattern)

> Derived from WAR bytecode (CFR). Narrative follows `DatabaseSqlProcessor.processData -> DatabaseSqlExecution.executeSQL`.

---

1. Conceptual model

```
Transaction XML (SQLXML)  OR  Write Query builds SQLXML in-memory
        |
        v
 getHandler() -> SQLXMLHandler (single or pooled JDBC connection)
        |
        v
 handler.execute(data)  -> DB reads/writes, XML response on stream
        |
        v
 returnHandler()  (pool) / keep singleton handler
```

```
|
+-- optional CallbackListenerName -> TriggerableListenerUtils.trigger(...)
```

The processor **replaces or transforms** the transaction stream with SQLXML results unless a callback listener consumes it.

---

2. High-level behavior

2.1 Processor entry (`DatabaseSqlProcessor.processData`)

1. `getSqlExecution().executeSQL(data, this)`
2. Return the same `TransactionData`.

2.2 Execution (`DatabaseSqlExecution.executeSQL`)

1. If **`WriteQuery`** is true -> `buildSimpleSQLXML(data)` replaces stream with generated SQLXML from **SQL** + **QueryParams** table.
2. **`getHandler(data, module)`** — obtains or creates `SQLXMLHandler` (non-pooled singleton `_handler` or pooled instance).
3. **`handler.execute(data)`** — runs SQLXML against the database; updates transaction stream with results.
4. **`returnHandler(handler)`** — returns pooled handlers; singleton path no-ops return.
5. If **`CallbackListenerName`** is non-empty -> **`returnResponse`** triggers the named **Programmable (Trigger)** listener with the result stream.

2.3 Shutdown

`DatabaseSqlProcessor.systemShutdown()` -> `shutdownHandler()` closes JDBC connections on the singleton handler.

---

3. Configuration reference

Configuration is defined in **`DatabaseSqlExecution.getConfigurationDescriptor`**, merged with inherited **`ExecuteProcessor`** from `AbstractProcessor`.

3.1 Query mode

UI label	Tag	Type	Default	Tooltip / notes
Write Query	`WriteQuery`	Boolean	`false`	Enables writing simple SQL queries (will replace transaction body with SQLXML)
Input file	`InputFile`	File	`""`	If specified, a SQLXML input file is executed. Hidden when Write Query is true

UI label	Tag	Type	Default	Tooltip / notes
SQL	`Query`	Text	`""`	SQL query to be executed (Write Query mode)
Query Params	`QueryParams`	Table	empty	Parameters used (in order) with the query

Nuances:

- **Write Query true** — ignores **Input file**; builds DOM via `CommonDatabaseUtils.buildSimpleSqlXmlQuer` serializes to bytes, `data.setDataStream(...)`.
- **Write Query false** — SQLXML comes from transaction body and/or **Input file** on the handler.

3.2 Connection (Connection tab)

UI label	Tag	Type	Default	Notes
Type	`UseDataSource`	Trigger	JDBC Connection (`false`)	Toggle: **Data Source** vs **JDBC Connection**
User name	`UserName`	String	`""`	JDBC mode
Password	`Password`	Password	`""`	JDBC mode
Data Source	`DataSource`	DataSource	`""`	Shown when Type = Data Source
JDBC driver	`JdbcDriver`	JDBCDriver	`""`	Required for JDBC mode (with URL)
JDBC URL	`JdbcURL`	String	`""`	Required for JDBC mode
Test Connection	(button)	—	—	Action `TestSqlConnection`

Configuration completeness: Data source name **or** (driver **and** URL) must be set.

3.3 Advanced tab

UI label	Tag	Type	Default	Notes
Keep connection	`KeepConnection`	Boolean	`false`	JDBC mode only; close between uses when false

UI label	Tag	Type	Default	Notes
Autocommit Transactions	`Autocommit`	Boolean	`false`	Each SQLXML document is a transaction when false
Enable Timeout for Queries	`EnableTimeout`	Boolean	`false`	Gates timeout item
Timeout for Queries	`Timeout`	Time	`600`	Seconds; aborts long queries
Disable Metadata	`DisableMetadata`	Boolean	`false`	Skips metadata load in handler
Restrict Metadata To Catalog	`RestrictMetadataToCatalog`	String	`""`	Performance filter
Restrict Metadata To Schema	`RestrictMetadataToSchema`	String	`""`	Performance filter
Restrict Metadata To Table(s)	`RestrictMetadataToTable`	String	`%`	Default `%`
CallBack listener	`CallbackListenerName`	ListenerName	`""`	Programmable (Trigger) listener
Use Single Output Stream	`UseSingleOutputStream`	Boolean	`false`	Single combined query response stream
Error on Unknown Elements	`ErrorOnUnknownElement`	Boolean	`false`	Fail on unknown SQLXML functions
Use JDBC Identity for Inserts	`UseJdbcGeneratedKeys`	Boolean	`true`	Overrides custom IdentityQuery in SQLXML when true

3.4 JDBC Properties tab

UI label	Tag	Type	Notes
JDBC Properties	`JDBCProperties`	Table (name/value)	Passed as `Properties` on connection

3.5 Pooling tab

UI label	Tag	Type	Default	Notes
Use Connection Pooling	`UseConnectionPooling`	Boolean	`false`	Multiple connections

UI label	Tag	Type	Default	Notes
Connections allowed	`PermittedDBConnections`	Integer	`5`	Max simultaneous handlers

Pooling algorithm: Waits in 100 ms loops until `currentHandlersInUse < PermittedDBConnections`; reuses handlers from a per-connection-string stack.

3.6 Debug tab

UI label	Tag	Type	Default
Log Metadata	`LogMetadata`	Boolean	`false`
SQLXML Logging	`LogSQL`	Boolean	`false`

3.7 Compatibility tab

UI label	Tag	Type	Default
Use JDBC Parameter Typing	`UseJdbcTyping`	Boolean	`false`

When true, statement parameters are type-checked by the JDBC driver only.

3.8 Transaction Isolation tab

UI label	Tag	Type	Default
Transaction Isolation	`TransactionIsolationLevel`	Multiple choice	first choice
Driver Specific Level	`TransactionIsolationDriverSpecific`	Integer	`-1`

Driver Specific Level visible when isolation choice is **Driver Specific**. Values from `CommonDatabaseUtils.TRANSACTION_ISOLATION_LEVEL_CHOICES`.

3.9 Inherited AbstractProcessor

Tag	Default	Tab
`ExecuteProcessor`	`true`	Conditional Execution

---

4. Runtime algorithms

Topic	Behavior
Handler reuse (no pool)	Single `_handler` reinitialized each execute
Handler pool	Keyed by connection parameters string; push/pop stack
Timeout	Applied when `EnableTimeout` true via `handler.setTimeout`
Metadata constraints	Catalog/schema/table patterns passed to handler
Callback	After execute, if listener name set, triggers with current stream
Errors	Wrapped as `EIPException("Error executing transport", cause)` — message text inherited from transport-era code
Write Query failure	`EIPException("Could not load simple SQL", cause)`

---

5. Worked example — parameterized lookup

Write Query: true
SQL: `SELECT status FROM orders WHERE id = ?`
Query Params: row value `%OrderId%`

The processor builds SQLXML, executes, and replaces the transaction body with XML results. Downstream XPath can read column values.

---

6. Known quirks and caveats

1. **Exception message says "transport"** — historical wording from shared SQL execution class.
2. **Singleton vs pool** — non-pooled mode keeps one handler per processor instance for the JVM lifetime until shutdown.
3. **Autocommit false** — each SQLXML document is one DB transaction; commit/rollback semantics follow SQLXML handler rules.
4. **Callback listener** — must be type **Programmable (Trigger)**; fires after execute with result stream.
5. **Input file vs stream** — typical routes pass SQLXML on the transaction; input file is for static templates.
6. **Generated keys default true** — custom SQLXML identity queries ignored when `UseJdbcGeneratedKeys` is true.
7. **Configuration title** — internal descriptor title `DatabaseSqlConfiguration` (shared with listener).

---

7. Operational recommendations

Goal	Guidance
Performance	Restrict metadata tables/schemas in high-volume routes
Pooling	Enable when many parallel transactions hit the same DB
Timeouts	Enable for user-facing routes to avoid hung JDBC calls
Secrets	Prefer data sources over embedded passwords in interface XML
Testing	Use **Test Connection** before deployment
Callback pattern	Use callback listener when results should fork to a sub-route
Simple queries	**Write Query** mode avoids hand-authoring SQLXML for one-off SELECTs

---

8. Source index (this WAR)

Topic	Path under `war-pipeline/decompiled/eip-26R1.11/`
Database (SQL) processor	`com/pilotfish/eip/modules/db/DatabaseSqlProcessor`
SQL execution engine	`com/pilotfish/eip/modules/db/DatabaseSqlExecution`
AbstractProcessor	`com/pilotfish/eip/extend/AbstractProcessor.java`
Module JAR	`war-pipeline/jars/eip-26R1.11/xcs-modules-db-1.0.`

---

9. Pipeline provenance

```
PilotFish WARs/eip.war.hs.26R1.11
  -> xcs-modules-db-1.0.1.jar
  -> CFR decompile
  -> Documents/Processors/26R1.11/PilotFish-Database-SQL-Processor-Reference-26R1.11.(md|pdf)
```

Deep-dive documentation from WAR decompile – Database (SQL) Processor – 26R1.11 – August 2026.